Independent Branches for Verification Serving

Anonymous research draft

Abstract

Formal verification services check related program and proof candidates that share libraries, declarations and accepted context. Serving independent attempts requires balancing repeated preparation with shared execution, whose resource growth, contention and cancellation depend on candidate isolation. Native execution history includes scope, heuristics, runtime objects and resource counters beyond the logical context, requiring a separate argument for observable verification behavior. Established result caches, persistent environments, proof-state snapshots and bounded warm services recover preparation reuse, while the value of independent native execution at matched resources remains unresolved. We argue that reusable verification state should retain preparation while separating each candidate's execution history and lifetime from preceding alternative attempts. Realizing this separation requires supported quiescent runtime boundaries, faithful native configuration and resource handling, and bounded memory as candidates modify branch-local state. Our verification service retains immutable prepared states in a parent, executes disposable independent branches with inherited native resource state and separate service accounting, and uses native execution for unsupported boundaries. Using public Lean and satisfiability modulo theories (SMT)-backed candidate streams, we compare complete verification against competent bounded warm services and native logical or serialized prepared-state reuse, then assess remaining heterogeneous checking after preparation reuse on an RTX 5090 with setup, transfer and CPU fallback charged. [**RESULT: End-to-end latency, throughput, memory and behavior results against the strongest matched services, including the measured scope of device benefit.**]

1 Introduction

Formal verification services check streams of program and proof candidates against formal specifications. Related attempts from proof-generation systems share libraries, declarations and accepted context. A proof assistant constructs and checks proof terms, while a program verifier translates obligations into requests to satisfiability modulo theories (SMT) solvers. Community tools for Lean [3] expose persistent environments for these workloads [11].

Serving independent attempts requires balancing repeated preparation with shared execution. Starting a separate verifier for each candidate repeats imports and parsing and replicates resident objects. Retained workers and shared logical environments avoid that repetition, but their resource growth, contention and cancellation behavior depend on worker isolation. Concurrent attempts against one imported Lean context, for example, need the same context but have independent lifetimes. Cancelling one attempt must release its speculative work without ending the others.

Native execution history makes this balance depend on more than visible formulas or declarations. An additional scope can change the solving path selected by Z3's combined solver [40], and earlier requests can alter heuristics and resource counters. A proof assistant also retains elaborator state and mutable runtime extensions beyond its immutable declaration environment [3, 11]. A logically equivalent reconstructed context therefore requires a separate argument for observable verification behavior.

Established reuse recovers preparation, leaving unresolved the value of independent native execution at matched resources. Result caches avoid repeating completed work [19], and the community read–eval–print loop (REPL) provides persistent environments, backtracking and pickling [11]. Lean snapshotting retains elaborated proof state across parallel tactic branches [36], while stock Lean saves incremental elaboration snapshots between invocations [13]. Bounded warm services reuse environment roots and retire workers [32, 34]. Axle serves preloaded Lean contexts in separate request processes [38]. A matched comparison against these native-reuse baselines must establish whether separating preparation from candidate execution history improves complete verification, cancellation and total memory across service processes.

We argue that reusable verification state should retain preparation while separating each candidate's execution history and lifetime. Preparing context before candidate evaluation makes that native state available to independent attempts. Each candidate inherits the prepared state, so preceding alternatives neither supply its solver history nor extend its lifetime. The benefit depends on preparation cost, branch creation cost and the pages that each candidate modifies.

Realizing this separation requires runtime safety, faithful resource handling and bounded memory. The branch boundary must be quiescent and safe for the verifier run-

time. Native configuration and resource accounting must remain faithful when process creation changes timers or descriptors [21]. Sharing must also improve service capacity within a memory budget that bounds retained states. These requirements connect compatibility to the service's resource frontier, which comprises achievable combinations of latency, throughput and memory use at matched CPU capacity and memory limits.

We present a verification service whose parent retains immutable prepared states for disposable independent candidate branches. Each branch owns its responses, mutable state, communication and cancellation, inherits its native resource state, and receives separate service accounting. The service addresses runtime compatibility and memory use through supported quiescent boundaries, untouched candidate operations after preparation and bounded prepared-state lifetimes, and uses native execution for unsupported boundaries. We compare complete verification on public Lean and SMT-backed candidate streams against competent bounded warm services and native logical or serialized prepared-state reuse. After preparation reuse, we assess remaining heterogeneous checking on an RTX 5090, charging setup, transfer and CPU fallback to complete verification. **[RESULT: Matched behavior results and end-to-end latency/throughput/memory findings, including the measured scope of device benefit.]**

This paper makes four contributions.

- A characterization of sharing expensive preparation in actual ordered verification attempts (Section 6.2).
- A native branching design and an argument about observable behavior separating preparation from candidate interference (Section 4).
- A bounded implementation integrating independent branches with native verification and cancellation (Section 5).
- An equal-resource evaluation against established reuse, with a causal assessment of heterogeneous checking after preparation reuse (Section 6).

2 Background

2.1 Verification stages and outcomes

Lean source verification includes parsing, elaboration, proof construction and kernel checking [3]. Imports prepare declaration environments before a candidate is elaborated, while kernel checking validates proof terms against the dependent type theory [3]. By contrast, post-export checker replay begins after the source frontend has already run. These boundaries distinguish savings in candidate construction, library setup and proof checking [14, 11].

SMT-backed verification includes frontend translation, solver setup, parsing, preprocessing and solving [10, 40]. A native SMT session can contain multiple queries and output requests. Solver responses include SAT for satisfiable queries, UNSAT for unsatisfiable queries, and UNKNOWN when the solver does not decide the query. Clients can also request models, proofs or unsatisfiable cores when the configured backend supports them [40]. These outcomes and artifacts define the comparison in Section 4.2.

An accepted Lean command may admit an unproved obligation using `sorry` [11]. Complete proof verification distinguishes unfinished proof obligations and admissions from frontend diagnostics.

Closing a tactic's goals is also distinct from checking the enclosing modified declaration. LeanPolish tests tactic proposals against saved proof states before re-elaborating selected source edits [1]. Its native menu screen restores state without separately finalizing each alternative as the original declaration [18]. A successful tactic screen therefore establishes a different outcome from acceptance of the enclosing declaration or source.

2.2 Prepared native state

A native execution prefix is an ordered sequence of verifier operations together with its configuration and runtime state. For SMT, this includes options, declarations, assertions and scopes in their observed order [40]. For Lean, it includes imports and accepted commands that establish the environment or proof state before the next attempt. Persistent tools provide ways to retain some of this state [11, 36].

Native interfaces retain verification state, while runtime and resource properties determine how that state can be reused. The solver's own command interface saves state across evaluations [40], and a resource limit can bound internal work independently of elapsed time [41]. Process branching shares existing memory through copy-on-write, but child timers, descriptors and runtime threads need separate handling [21]. Section 4 uses these native-state and process properties to define preparation boundaries.

3 Motivation

3.1 Preparation and candidate lifetime

A service must account for preparation reuse and candidate lifetimes. Preparation has value when attempts reuse expensive state, rather than merely share similar text. **[RESULT: Final characterization of preparation CPU cost, memory and candidate fan-out from real Lean and SMT-backed streams.]** A service must also account for attempts that are rejected, remain incomplete or exhaust their resources. Their lifetimes consume capacity even when the client seeks only one accepted candidate.

Established persistent sessions make cold process execution an insufficient performance baseline. The community REPL already allows multiple commands to refer to an earlier environment [11], and native proof-state snapshots support parallel tactic exploration [36]. We ask whether sharing native preparation changes concurrency, cancellation and aggregate memory relative to bounded versions of these services.

3.2 Scope and history effects

Logical equality of visible assertions does not establish equality of the native state. The distinction between logical context and execution history is observable in resource-bounded verification. [**RESULT: Final native-session and prepared-branch comparisons, including scope-only and prior-history controls for status differences.**] Scope effects can select a different solver path even in a fresh process [40], while history effects can arise without a change to the candidate's explicit assertions. A compatibility argument must distinguish both causes from timing variability and account for native resource and runtime properties when establishing a faithful branch boundary.

The preparation/isolation tradeoff and scope/history distinction motivate the design in Section 4. Preparation sharing, candidate isolation and bounded resource use are evaluated together. A speedup from a path that changes the verifier responses or requested artifacts is excluded from the service comparison, even when that path remains logically sound on some inputs.

4 Design

The design has three goals derived from Section 3: (1) preserve the native verifier's observable behavior, (2) share expensive preparation while isolating candidate lifetimes, and (3) improve latency and throughput within matched CPU and memory resources. The service treats a prepared state as an immutable branching point. Candidate evaluation occurs only in an independent branch, and unsupported boundaries use native execution.

Placement first chooses whether retaining preparation is worthwhile, and requests without a retained state use native execution. For a selected prepared state, boundary qualification separately determines whether branching is supported (Figure 1). The parent owns the immutable prepared state, while each supported branch owns candidate mutation and its lifetime. Supported branches execute on CPU with supported device operations where available, while unsupported boundaries use native execution.

4.1 Preparation boundaries

A shared parent must represent state that every assigned candidate has available. The service therefore matches

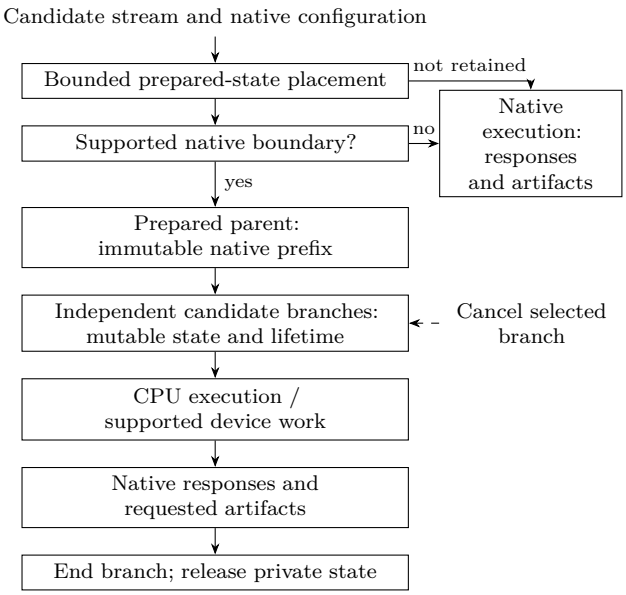


Figure 1: Supported branches inherit an immutable prepared prefix and own candidate mutation and lifetime. The parent performs no candidate verification after the selected boundary. Cancelling a selected branch ends its private work while leaving the parent and other branches independent. Normal completion releases private state after returning native responses. Unsupported boundaries use native execution. Placement can also select native execution without retaining a prepared state.

the original ordered preparation and configuration, without replacing them with a set of assertions. A candidate suffix begins after this preparation and includes its original scopes and output requests. The parent retains accepted context in the original preparation, but contains no learned or elaborated state produced by candidate suffixes after the selected boundary.

The boundary must also be safe for the runtime. Each adapter exposes a supported quiescent point with no pending candidate work or unresolved external effects. The service retains the prepared objects and initializes branch-owned communication and resource controls before evaluation. A runtime without such a point follows the native path. This restriction is a deployment boundary, not evidence that arbitrary native processes can be cloned safely.

4.2 Observable behavior

Native behavior is defined at the verifier interface, independently of the branch implementation. The reference is original native execution of the same ordered preparation and candidate suffix. This preparation retains the candidate's accepted context without inserting suffixes from preceding alternative candidates. History within the candidate's own native session remains part of its suffix.

Behavior comparisons cover verifier outcomes and requested artifacts. For Lean, the comparison includes diagnostics, completed proof obligations and admissions in each attempt. For SMT, it includes each SAT, UNSAT or UNKNOWN response and each requested artifact. Native proof or model validation checks artifact meaning where alternative valid artifacts can differ in representation. Counting accepted commands alone does not determine fidelity between verification paths.

The candidate executes its original suffix once in a branch. The service inserts no additional logical scope and performs no preparatory solver check to warm heuristics. Native solver resource state is inherited from preparation, while service CPU and elapsed costs are charged separately. These rules remove cross-candidate history as a source of difference, but wall-time limits and changed runtime conditions still require measurement rather than a universal assertion that responses remain equal.

The service chooses fallback according to interface compatibility, rather than treating every UNKNOWN response as a detector of divergence. Returning UNSAT when the native resource-bounded path returns UNKNOWN can also change client behavior. The service selects native execution when the adapter cannot support the boundary or requested artifact. Any residual mismatch in a supported comparison invalidates that path's fidelity claim. The service retains the mismatching case with its complete input and output.

4.3 Branch lifetime and cancellation

Candidate mutation should not force the service to retain the shared parent beyond its configured policy or affect another candidate. Each branch owns mutable candidate state and terminates after producing its response, while cancellation stops that branch and releases its private memory. Device work and buffers remain attributable to their candidate until completion or cancellation cleanup, which is included in branch lifetime and service accounting. The prepared parent remains available only while it satisfies the configured retention policy.

The lifecycle separates retained preparation from speculative work. A cancelled or incomplete attempt cannot become a new prepared parent. Branch failure returns an explicit native or service failure and leaves other branches independent. The evaluation includes failed attempts and cancellation recovery rather than reporting only successful candidate throughput.

4.4 Bounded placement and execution

The benefit of preparation sharing must offset the costs of retained memory and branch mutation. The service admits prepared states within a memory budget and accounts for proportional shared memory and private dirty pages. Proportional set size (PSS) attributes each shared page across the processes sharing it [20], and fleet memory sums these attributed amounts across service processes. Placement favors states whose observed preparation work is reused often enough to offset the costs of creating and retaining those states. Branch concurrency is limited by CPU capacity and memory pressure, so copy-on-write sharing does not justify unlimited fanout.

If candidates modify most retained pages, copy-on-write sharing can approach full replication. If preparation is cheap or locality is low, native execution avoids retention overhead. These cases test the same service tradeoff as workloads with substantial preparation sharing and define its measured operating range.

4.5 Heterogeneous checking

When preparation reuse removes repeated work, it leaves a smaller verification path for an accelerator to improve. The service selects expensive supported operations from profiles of that path and compares them with the strongest CPU implementation. For these checking operations, the input expression graph alone does not define dependencies, because reduction depends on binder environments and instantiated constants [3, 16]. A GPU operation therefore includes its actual dynamic work, required representations and synchronization.

Supported device work preserves verifier semantics. Before a device operation is admitted, device qualification compares its outputs with matched CPU references. During execution, enclosing native verification checks the resulting proof or requested artifact where one is produced, and required online checks are charged to complete verification. Qualification does not imply a second full CPU replay on every request. Response and artifact fidelity follow Section 4.2, including UNKNOWN and responses without artifacts.

Unsupported work returns to CPU execution, whose cost remains in the complete verification measurement. Complete verification includes device initialization, representation construction, upload, download and queueing. We report operation speedups measured with data already on the device separately from complete checking.

5 Implementation

The implementation uses native verifier adapters with separate process-branch and shared-runtime state-branch constructors in a bounded service. Their behavior and performance results apply to the boundary and runtime version measured. The SMT adapter retains Z3's native command context across native parser invocations to evaluate ordered session state [40]. The Lean adapter uses the runtime boundary that prepares imports and accepted proof context before a candidate. For each verifier, qualification first compares fresh adapter execution with the original interface and then compares

prepared branches with that reference. We measure reuse after these comparisons.

5.1 Native adapters

The native SMT command adapter is implemented in C++. It retains the command-line interface's command context and preserves the native order of configuration, manager creation and solver creation while evaluating original command sequences. It preserves options, scopes, checks and requested artifacts rather than reconstructing assertions through a different solver API.

Qualification compares fresh adapter execution with the matched standalone executable, using an unmodified executable built from the same source to distinguish build metadata from other response differences. This comparison separates interface incompatibility from differences caused by branching. Every observed response difference remains visible. Diagnostic branching tests can investigate a rejected adapter, but the service executes prepared-state branches only with an adapter that passes interface qualification.

A preparation boundary can separate parser invocations while retaining the native command context. Qualification therefore compares complete input, split native parsing and branch evaluation to distinguish parser end-of-input effects from branch effects.

For source verification backed by this SMT adapter, the measured interval also includes the native frontend's translation and its original solver interaction. Solver-session replay alone is reported at its narrower boundary.

For Lean, the adapter retains accepted context and keeps branch-local proof construction independent. It waits for source snapshots and the checked declaration environment before qualifying runtime quiescence, thread ownership and mutable extensions. Frontend completion alone does not establish that runtime workers, their finalizers and event loops have stopped. The adapter enables native process branching only after that separate runtime qualification.

5.2 Shared-runtime native state

The shared-runtime Lean constructor retains the prepared native command state while assigning each candidate a separate mutable-reference context. Reads of prepared reference cells use the retained base until a destructive operation creates a private cell. Unresolved prepared thunks are evaluated privately. Branch identity follows admitted native tasks, and branch release waits for those tasks and their execution scopes before releasing retained identities and private versions.

This reference overlay does not make arbitrary native globals, extensions, external effects or foreign-function interfaces (FFI) private. Listed guarded effects select native replay. When unsupported effects terminate a worker, the operational outcomes remain recorded. These mechanisms establish neither general process equivalence nor completed cancellation qualification. The process constructor in Figure 1 and this runtime constructor have different supported boundaries and isolation properties.

5.3 Process and resource handling

Linux copy-on-write branches retain the parent address space, with candidate writes creating private pages [21]. The parent stays quiescent and never executes candidate suffixes, while branch communication uses separate descriptors. The service reaps children after response, cancellation or failure and distinguishes native solver resource counters from service elapsed and CPU accounting.

Cancellation instrumentation records the native progress reached before termination and whether the child was reaped before later work. The instrumentation establishes reaping after native progress, while evidence of interruption inside a check requires separate execution-phase records.

Qualification of supported boundaries covers native resource controls affected by process creation. Claims about wall-time outcomes are limited to the measured operational scope described in Section 4.2 and the evaluation limitations, rather than following from inherited counters alone.

The service bounds prepared-state retention and branch concurrency using the preparation-reuse criterion in Section 4.4. Instrumentation records parent and child proportional memory, private dirty memory and process lifetimes. This accounting distinguishes shared preparation from full worker replication and includes replacement costs.

5.4 Device path

The device path uses an allocated RTX 5090 through NVIDIA's container runtime [30]. During device qualification, a CPU reference and GPU implementation consume the same identified input and preserve output identity for each supported checking operation. Required online checks follow Section 4.5. Native artifact fidelity follows Section 4.2, and complete verification includes CPU fallback and device/runtime costs. For checker representation, existing techniques include expression layouts with integer identifiers and delayed substitution [16, 15], so their use does not establish a new checking algorithm.

The Linux implementation integrates native SMT session evaluation in C++, Lean context reuse and matched CPU/device execution through these adapters.

6 Evaluation

The evaluation answers four research questions, each mapped to the design goals and contribution it tests.

RQ1 tests the sharing opportunity characterized by the first contribution. RQ2 tests behavior preservation and isolation, the first two design goals, while RQ3 tests the third goal, the equal-resource service frontier. RQ4 tests whether heterogeneous execution improves that frontier for complete verification after reuse.

RQ1 How much expensive native preparation can real verification candidate streams share across independent jobs?

RQ2 Does branching from prepared native state preserve verifier responses and requested artifacts while isolating candidates and cancellation?

RQ3 How does prepared-state branching change end-to-end latency, throughput and memory at equal resources compared with established warm-session reuse?

RQ4 After preparation reuse, when does heterogeneous execution improve complete verification over the strongest measured CPU path?

6.1 Experimental setup

We use complete ordered public proof and program attempts from multiple contexts in Lean and SMT-backed systems verification. Published proofs and deliberate invalid or incomplete variants provide behavior controls, while failed generated attempts remain part of the candidate-stream evaluation. These controls do not establish locality in generated candidate traffic. VeruSAGE provides real repository-level tasks [26, 39] for Verus [10], and Mathlib and the Lean Kernel Arena provide source/library and checker workloads [12, 14]. **[RESULT: Final source-native candidate-stream identities, versions, complete coverage and exclusions.]**

The SMT comparison replays 3,566 retained Rust candidates from the released AutoVerus artifact [27] in source-sorted order. Saved outputs include intermediate and unsuccessful attempts. This order does not reconstruct historical verifier arrivals.

Retained generated candidates and submission policy. The retained Goedel-Prover-SFT generation [5] contains 32 candidates for each of 244 miniF2F test tasks [42] and 186 ProofNet-lineage test tasks [33, 6], for a total of 13,760 candidates. Every selected candidate maps to its original task, model output, extracted source and submission/response events in four RTX 5090 generation segments. Interrupted tails and 24 additional returned responses outside the complete 32-candidate groups remain retained separately. The adapted SGLang [35] generator executes one request at a time. Official Goedel inference instead batches model inputs through vLLM [5, 37]. These event clocks record candidate availability, rather than verifier-request arrivals. The official next stage reads the completed code file and submits all candidates as a batch [4]. Consequently, finite bulk verification and online offered-load replay have distinct submission policies. Our whole-source CPU comparisons load all inputs before service startup and use four outstanding HTTP requests. They are not exact reproductions of the official default 64-request scheduler.

Version-aligned released proof workload. We also characterize all 29,750 released GoedelWorkbook proof files migrated to Lean 4.27.0 and Mathlib revision a3a10db0 [28]. The retained metadata has 29,750 distinct problem identifiers, with one source per identifier. This cohort does not establish repeated same-problem preparation savings. We retain every published source file, including the publisher's migration errors and timeouts, in filename order. This release contains proof outputs, rather than every attempted generation or observed service arrival. Its competition-mathematics focus limits generalization to other Lean use.

One complete native-compilation reference acquisition uses four outstanding HTTP requests, shared CPU eligibility 0–3 and the pinned Kimina service with a 300-second request timeout and 12 GiB per-process address-space allowance. Candidate requests use its garbage-collection (GC) mode, which does not retain their command snapshots. It returns 27,319 complete, 2,376 rejected and 55 operational outcomes. The operational group contains 41 timeouts and 14 decoder failures. An individual native allocation-failure replay does not establish the cause of every decoder failure.

The main service competitors are bounded warm services with their existing cache and environment mechanisms, and native logical or serialized prepared-state reuse [32, 11, 13]. Existing REPL environment backtracking and native logical proof-state branching remain baseline implementations. Fresh native execution supplies the behavior reference and a preparation control. Baselines receive the same candidate information, CPU capacity, memory budget and admission order.

For the matched Lean 4.27 native-batch comparison, the primary competitor is the existing Kimina naive `BatchCommand` path [32], which runs native tasks from one prepared root. Disabling mutable-reference branching in the candidate method is a separate component control. All methods return an ordered batch array after its last member, so batch emission is not compared with streaming completion as a latency advantage.

The earlier eager-copy whole-corpus acquisition is stopped with its complete control and partial candidate data retained. The subsequent first-write runtime has finite native engagement and matched diagnostic checks, but no stable advantage over stock. This runtime constructor is closed; neither version provides a completed comparative performance result. The native-batch ex-

periment inherits the hosting environment's address-space policy after all methods fail under the service's 12 GiB policy. It therefore does not establish equivalence to the memory policy of a default Kimina deployment.

The heterogeneous comparison uses the fastest runnable complete CPU checker at a matched boundary, in addition to official Lean. A kernel checker operates after export only when the measurement explicitly uses that boundary. [**RESULT: Pinned strongest service artifacts and matched configuration.**]

We preserve all raw per-input outcomes. Lean outcomes include unfinished proof obligations and admissions as well as frontend diagnostics. A successful tactic screen requires the enclosing declaration or source checking stage before it counts as an accepted proof.

Primary metrics are end-to-end candidate latency distributions, completed attempts per second, corpus makespan, CPU time and fleet proportional memory. Queueing, preparation, parsing, elaboration, proof construction, checking, transport, setup and teardown are included where they occur. Throughput cost per completed job is distinguished from request latency.

We repeat whole comparisons with alternating condition order. Affinity specifies permitted CPUs but alone does not provide CPU isolation [23], so we record affinity with topology and observations of competing work. [**RESULT: Hardware, topology, software versions, repetitions, observed load, uncertainty and measurement details.**]

6.2 Preparation sharing

RQ1 asks how much expensive native preparation real verification candidate streams share across independent jobs. We preserve original attempt ordering and measure preparation CPU time and resident state available before each attempt. We separately analyze result reuse for exact duplicate attempts, sharing of ordered native prefixes and sharing of native environments. Sharing a native environment characterizes established reuse, while a supported process branch must also satisfy the preparation and boundary conditions in Section 4.1. The characterization includes failed and incomplete attempts, so locality is measured across all attempts rather than only successful proofs.

A complete source-level characterization finds one original task header in the miniF2F pool and 17 in the ProofNet-lineage pool. Apart from extraction-failure strings, the pools contain zero and 121 repeated code occurrences, respectively. Header recurrence and source duplication do not measure native preparation CPU or memory. In the complete 29,750-source modern GC captures, the original service executes 67 import headers and selects 29,683 started-REPL reuses. The compiled route executes 60 headers and selects 29,690 reuses. All source IDs are accounted for once. These counts confirm that the competent baseline already reuses preparation, but do not measure its CPU or latency share. Older preparation controls also include an explicit `LibrarySearch` query for `True`. Their initialization intervals therefore do not establish preparation demand derived from the candidate requests.

A separate request-driven observation measures all 5,952 ProofNet-lineage candidates on the producer-compatible Lean4.9 GC and selective native arithmetic service, without the injected search query. Stock header splitting maps 5,947 requests to one `import Mathlib` key and leaves five literal extraction-failure strings with empty headers. In each of three whole observations, the service performs five nonempty import preparations and 5,942 started-worker reuses. Preparation consumes 12.31, 12.99 and 12.75 observed native CPU seconds; the corresponding observed lifetime totals are 1,522.98, 1,531.77 and 1,521.46 CPU seconds. Their ratios are 0.81–0.85% of observed native CPU, not an exhaustive CPU fraction or an end-to-end speedup bound. Each observation returns 413 complete, 5,534 rejected, four incomplete and one operational outcome in 468.27, 470.37 and 465.68 seconds, respectively. The failed worker's unobserved CPU tail remains censored. Empty-header initialization can occur in its first body and is not claimed free. These observations do not establish consequential remaining import preparation cost after competent reuse on this cohort.

A phase reanalysis of the complete SMT streams measures the cost paid to prepare a reusable parent. Each branch observation prepares 473 parents across 136 prefix identities. Their cumulative CPU from process birth to readiness totals 7.568, 7.636 and 7.712 seconds, with per-parent medians of 14.9, 14.8 and 15.2 milliseconds. Logical reuse totals 7.699, 7.571 and 7.537 seconds under the corresponding three primary observations. These counters include native initialization before parsing and exclude subsequent candidate suffixes, the Verus frontend, transport and service management. They are neither a whole-verification CPU fraction nor measured avoided work. In particular, 2,316 parent hits do not imply 2,316 avoided expensive preparations: logical reuse still reparses and translates each candidate. This modest absolute readiness cost supplies no reason to reopen the slower process-branch constructor in Section 6.4.

We compare original ordering with a control containing unrelated work and measure how preparation depth changes reusable work. Text bytes and declaration counts are descriptive inputs, while measured preparation cost determines the opportunity. [**RESULT: Preparation-cost distributions across remaining contexts, context reuse over time and aggregate achievable sharing with uncertainty.**] RQ1 remains unanswered until these measurements of complete original candidate streams establish whether preparation sharing is consequential.

6.3 Behavior and isolation

RQ2 asks whether branching from prepared native state preserves verifier responses and requested artifacts while isolating candidates and cancellation. We first compare fresh adapter execution with the original verifier, then execute untouched candidate suffixes from prepared states. We retain per-input decisions and artifacts for comparisons that vary preceding unrelated attempts, branch cancellation and resource-limit conditions. Completed attempts are compared for native responses and artifacts. Externally cancelled attempts are compared for progress at termination, cleanup and the independence of surviving attempts. Native scope and history controls distinguish changes in solver mode from cross-candidate interference.

Observed behavior within the completed service comparisons. The SMT comparisons in Section 6.4 preserve native inputs, basic results, decision sequences and requested artifacts on their reported primary and resource conditions. The ProofNet service replay preserves basic outcome counts, whereas the shared-process placement retains diagnostic differences in the numbers assigned to metavariables, which are placeholders used during elaboration, on incomplete sources. These comparisons use distinct constructors and comparison scopes. Basic outcome agreement does not establish identical response bytes or general artifact fidelity. Section 6.4 retains their timing and resource conditions. The remaining history, resource-limit and cancellation comparisons below still determine RQ2 at its full scope.

Lean controls cover complete acceptance, rejection, unresolved obligations and admissions. SMT controls cover SAT with model requests, UNSAT with supported proof/core requests, UNKNOWN and multi-check sessions. The original scope/history regression streams remain part of the causal test. [**RESULT: Complete per-input comparisons, artifact validation, resource-limit outcomes and cancellation recovery.**] RQ2 remains unanswered until every planned behavior comparison finishes with a recorded outcome, residual differences are explained and divergent paths remain excluded from qualified service comparisons.

6.4 Latency, throughput and memory

RQ3 asks how prepared-state branching changes end-to-end latency, throughput and memory at equal resources compared with established warm-session reuse. We replay actual attempt streams over offered-load and concurrency sweeps, matching CPU and memory budgets across services. A single shared warm worker is not the sole comparison to many branch workers. We charge the measured service for creating and replacing prepared states and for cache costs.

We report latency distributions and completed

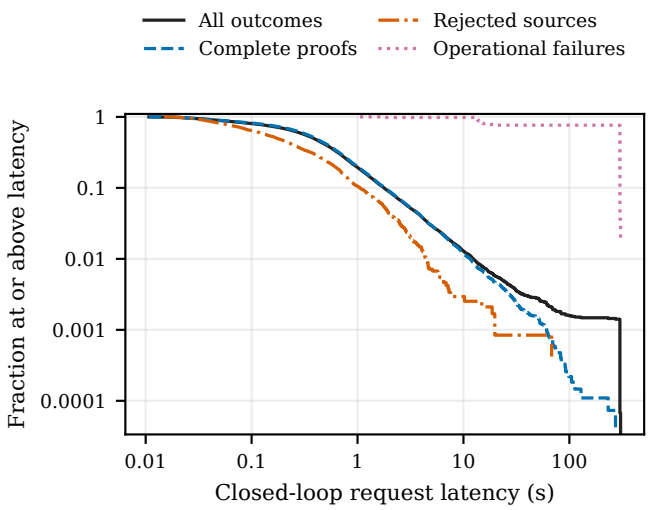


Figure 2: Request-latency survival curves for one complete 29,750-source native CPU reference acquisition. Each curve is normalized within its indicated outcome population. Both axes are logarithmic. Operational outcomes remain included. This single acquisition characterizes the workload tail and provides no between-run uncertainty, speedup or offered-load queueing result.

throughput together with fleet proportional memory and CPU work. The causal comparison includes startup-only branches, deeper prepared prefixes and independent warm workers. Ablations remove prefix preparation, vary preparation depth and replace placement with ordinary bounded retention. Private dirty pages and branch lifetime explain whether sharing or simple warm startup drives the observed gain. [**RESULT: Matched frontier plots, tail latency, memory, cancellation cost and mechanism ablations.**]

The comparisons below test distinct execution and placement mechanisms. Each result applies to its stated constructor and workload. The complete frontier comparison must also account for the full workload and resource matrix.

Single native-reference latency distribution. The GoedelWorkbook native-compilation reference uses the setup described above. Figure 2 separates these outcome populations. Completed proofs have median request latency 0.399 seconds, p95 3.324 seconds and a maximum of 272.692 seconds. These are closed-loop request durations, which overlap and do not decompose whole-corpus wall time or kernel CPU cost.

Complete ProofNet service replay. We replay all 5,952 retained ProofNet-lineage source attempts through a local native stream interface [33, 6]. The warm pool retains native workers across candidates, independent branches replace a worker after each uncached candidate, and snapshot/cache uses separately started native

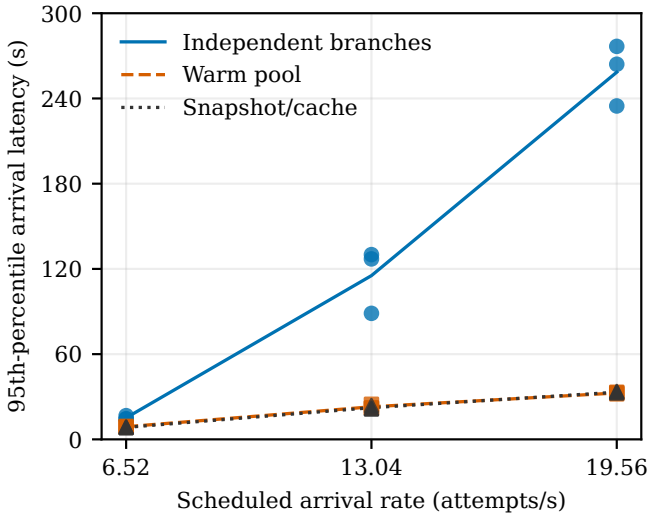


Figure 3: Arrival latency under synthetic load over 5,952 original ProofNet-lineage attempts per condition. Markers show all three whole-run p95 observations for each method and scheduled rate. Lines join their arithmetic means, not pooled request percentiles or confidence intervals. All 27 primary conditions complete on the same four permitted CPUs. Independent branches have worse p95 latency at each tested rate. The separate profiles are excluded. This finite replay does not establish a production arrival trace or steady-state capacity.

full-stream workers. All methods use the same success-only exact header/source cache policy, four worker slots and the same four permitted CPUs. They retain the Lean4.9.0-rc1/Mathlib environment, native runtime threads and original 300-second allowance from dispatch. Queue waiting does not consume that verifier allowance.

This retained protocol also explicitly warms a library-search index with a `True` query before candidate checking. Its observed wall interval is 6.69–6.93 seconds in the warm-pool and branch captures. The snapshot worker calls the same query without emitting a separate timer. These measurements therefore characterize this modified preparation protocol. They do not measure preparation demanded by the original requests or an untouched official service. No ablation isolates the query's effect on queueing or the method ratios.

Each method receives the original source order at deterministic synthetic arrival rates of 6.52, 13.04 and 19.56 attempts per second. Method and rate orders rotate over three complete primary repetitions. Three separate resource profiles bring the complete ProofNet cohort to 30 conditions. Every condition returns 413 complete, 5,534 rejected, four incomplete and one operational outcome, matching the retained stock basic outcomes. Arrival latency starts at the scheduled arrival and ends at captured response, including release lag and pending-queue time. Whole-corpus clocks additionally include initialization, final drain and end of file (EOF).

Figure 3 rejects a claim of faster service for this per-candidate branch implementation at the tested preparation boundary. At the highest rate, the branch implementation's p95 is more than seven times the warm pool's in each repetition. Low-rate whole times mostly follow the arrival schedule, so their similarity does not establish equal service capacity. At the two higher rates, branch mean waiting before dispatch is 41.658–142.095 seconds, compared with 0.363–0.395 seconds after dispatch. Mean release lag stays below one millisecond. These means locate waiting within the measured implementation. They do not isolate process-cloning cost or identify CPU-time fractions.

Separate profiles observe sampled PSS maxima of 2,764,774 KiB for branches, 4,806,597 KiB for the warm pool and 8,857,235 KiB for snapshot/cache. The branch profile has incomplete observations in 187 of 633 samples. One-second non-atomic sampling misses short-lived processes and peaks, so these single profiles do not establish replicated memory savings or a complete memory frontier. The wider workload, context and resource comparisons remain open.

Native SMT prefix comparison. The native SMT prefix comparison rejects acceleration for the tested constructor and cohort. Three complete rotations check all 3,566 retained AutoVerus source candidates through stock native execution, prepared native branches and logical reuse. The matched conditions use the same three permitted CPUs and original native configuration, charging startup, transport, raw capture, archival and cleanup. Stock whole times are 1,561.1, 1,618.0 and 1,889.7 seconds. Branch times are 2,124.1, 2,231.5 and 2,434.8 seconds. Branches are 28.84–37.91% slower and use 22.67–25.05% more CPU in the recorded counter.

The recorded CPU counter combines the acquisition process clock with `getrusage` child totals, which depend on completed waits [22]. Exhaustive descendant coverage is not established, so the ratios do not establish total process-tree CPU overhead.

All 10,698 stock/branch source comparisons preserve native inputs, basic results, decision sequences and requested artifacts. Each branch condition engages 2,316 retained-parent hits and 473 misses, so the measured elapsed-time and recorded CPU increases occur with actual reuse. Branch/logical ordering varies, and logical-stream completeness annotations remain in two rotations. All three separate resource captures also complete, with matching native input, results and requested artifacts across each pairing.

Separate resource captures observe sampled PSS maxima of 224,644 KiB for stock, 783,477 KiB for branches and 791,914 KiB for logical reuse. Between 233 and 368 samples per method contain incomplete reads. These single non-atomic profiles do not establish replicated or exact memory bounds. They show no stock-relative memory saving that compensates for the measured time loss.

Whole corpus time (s)	Run 1	Run 2	Run 3
Process-isolated warm pool	303.029	303.826	303.671
Shared-process alternative	371.637	360.690	349.614

Table 1: Complete native source verification for the shared-process placement and warm pool, each with 5,952 original attempts. The three primary pairs retain identical basic outcomes. Resource profiles are separate conditions.

The complete service frontier remains unfinished.

Complete source verification across placements. We compare a shared-process warm-worker alternative with the competent process-isolated warm pool on the retained 5,952 ProofNet-lineage source attempts [33, 6]. Both use the retained Lean environment and reuse native incremental snapshots, prepare the first imported environment and search index, and use four frontend workers on the same four permitted CPUs with default native runtime threads. The warm pool already shares its prepared roots through copy-on-write. The alternative shares one native heap and runtime while keeping each worker's root table separate as candidates update it. Complete clocks include initialization, preparation, source verification, response transport, actual failure recovery, output, EOF and cleanup.

Table 1 rejects an acceleration claim for this shared-process placement on the measured cohort. The shared-process alternative is 22.64%, 18.72% and 15.13% slower than the warm pool. Each condition returns 413 complete, 5,534 rejected, four incomplete and one operational outcome. A native panic ends the shared process. The complete method checks every unreturned original candidate through the ordinary native warm path and charges both phases. The primary pairs recover 13, 14 and 13 candidates in this way. Internal metavariable numbers in diagnostic suggestions differ on two retained incomplete sources in some pairs. These responses remain unnormalized, so matched basic outcomes do not establish identical artifact bytes in general.

Separate profiles observe sampled fleet PSS maxima of 7,156,560 KiB for the shared-process alternative and 7,574,628 KiB for the warm pool. The 5.52% lower sampled PSS is a single comparison, not replicated memory evidence. One-second non-atomic sampling and competing mappings limit PSS interpretation. A shared heap also changes whether unchanged anonymous pages are counted as private, so private-page totals across these placements do not directly measure mutation.

Response extraction after warm preparation. An optimization of response extraction yields a small complete verification improvement on the same retained ProofNet cohort. The producer REPL classifies explicit `sorry` nodes by printing an entire syntax node and comparing its string [11]. We add a necessary check of the shortened parser-node kind before the unchanged full-string predicate. The optimization retains the native tree traversal, goal snapshots, diagnostic construction, enclosing frontend and kernel.

Three alternating complete primary pairs use the same warm-pool host, prepared state, four permitted CPUs and native defaults. Original whole times are 303.124, 308.703 and 301.252 seconds, compared with 297.520, 301.563 and 299.071 seconds with the precheck. The observed wall reductions are 1.85%, 2.31% and 0.72%, with a descriptive mean of 1.63%. This observed range is not a confidence interval.

Each pair preserves all 5,952 basic outcomes. Requested responses are identical in two pairs. One source in the other pair has internal metavariable-number differences in diagnostic hints, which remain unnormalized.

Separate profiles contain complete timer sequences in both methods for 5,791 source indices. Over this subset, cumulative calling-thread CPU falls from 8.476 to 0.664 seconds for sorry extraction and from 20.170 to 12.296 seconds for complete response extraction. Interleaved or incomplete records are censored. These region totals exclude asynchronous frontend threads and subsequent response serialization and construction of auxiliary declaration records. These totals are not fractions of corpus wall time.

Separate resource profiles observe sampled fleet PSS maxima of 7,860,678 KiB for the original method and 7,728,536 KiB with the precheck, which do not establish replicated memory savings. This optimization strengthens a CPU comparison on the measured cohort. It does not establish systems novelty or heterogeneous benefit.

RQ3 remains unanswered until branching is compared with the strongest competent services over the complete workload, offered-load and resource matrix.

6.5 Heterogeneous execution

RQ4 asks when heterogeneous execution improves complete verification over the strongest measured CPU path after preparation reuse. We identify whether each complete measurement begins with source verification or post-export checking. Results from post-export checking therefore apply only to that checking boundary. We profile warm complete checking and identify costly operations and their actual context-sensitive dependencies. Device qualification runs the same supported operation on CPU and RTX 5090 with exact reference comparisons, while required online checks follow Section 4.5. All unsupported work remains in the complete verification interval.

We separate host representation construction, device setup, transfers, checking with data already on the device, synchronization and fallback. The complete CPU/device comparison uses the service accounting and matched workload boundary of RQ3. An improvement

in a checking operation alone therefore does not establish an improvement in the service frontier. The comparison includes tuned alternative CPU checkers with different parsing and reduction algorithms [16, 15]. **[RESULT: Cost coverage, amount of checking work simultaneously ready to execute, complete CPU/device comparisons and workload crossover.]**

Within the arithmetic CPU family, existing Farkas [25] provides native exact certificate search and numeric hypothesis selection before stock proof reconstruction. Our HiGHS support-search adapter uses the established linear programming (LP) solver [8], followed by exact certificate repair and Lean's standard proof construction and checking. These are CPU competitors in the heterogeneous comparison.

Here tiered denotes native exact certificate search, while hybrid denotes native double-precision (FP64) support search followed by exact repair. Fast/hybrid combines the upstream FarkasFast numeric probe and restricted stock proof construction with the native hybrid route and full native fallback [25].

Matched CPU competitors and controls. Table 2 summarizes the complete primary observations used to qualify CPU alternatives. Each comparison uses its own version, paired reference and CPU eligibility. Full clocks, native outcome changes, resource observations and sampling limitations remain in Appendix A. These supporting controls strengthen the CPU baselines used for comparison. They do not make ordinary compilation or invocation-local reuse the system contribution. The Goedel4.27 and miniF2F4.9 clocks cannot serve as reference measurements for the following ProofNet comparison.

Complete ProofNet source verification. Three complete primary comparisons process the same 5,952 retained ProofNet-lineage [33, 6] Lean4 source attempts in original order, including failed attempts. Each method uses four frontend workers on the same four permitted CPUs, the retained Lean4.9.0-rc1/Mathlib environment, default native runtime threads and the original 300-second per-attempt allowance. The native warm pool reuses prepared import headers. The native snapshot/cache condition retains full native snapshots and caches only complete results for exact matches of both import header and candidate source. The GPU method uses an RTX 5090 and includes conservative host admission, packet construction, device initialization, transfer, synchronization, CPU fallback, enclosing source verification and final native checking. Lossless archiving of experiment outputs occurs after the measured final EOF. CPU affinity does not provide exclusive cores. GPU time slicing shares the device with another Workspace.

Table 3 rejects an end-to-end acceleration claim for this supported expression reduction in Lean's metaprogramming (Meta) layer on the measured cohort. Relative to the faster native baseline in each block, GPU execution is 1.74%, 3.64% and 1.58% slower, with a mean of 2.32%. These values describe the observed repetitions, not a confidence interval. Every condition has 413 complete, 5,534 rejected, four incomplete and one retained operational outcome. The packed CPU version is a component control. It rejects unsupported operations at a different point from the GPU path, so their time difference cannot isolate device placement alone. The strongest native baseline grounds the end-to-end comparison.

Native operation/readback logs establish actual GPU engagement, while unsupported work remains native. The implemented provider launches one CUDA thread [29] for each supported beta or zeta reduction. These reductions substitute an argument into a function body or a value into a let-bound expression. The provider then synchronizes before returning to Lean. It does not batch independent reductions or execute the

CPU comparison	Whole elapsed versus its own paired reference
GC/native libraries, Goedel	51.93%,52.38%,59.52% lower; separate profiles unfinished
Selective native arithmetic, miniF2F	7.61%,10.74%,16.96% lower; mixed p99
Fast/hybrid product reuse	9.82%,12.79%,15.39% lower
Original-oracle product reuse	0.59% longer;5.11%,14.30% lower
Native exact oracle	4.73% longer;18.66%,1.40% lower
FP64 hybrid over tiered	10.61%,5.22%,23.43% longer
HiGHS over tiered	11.83% longer;11.45% lower;21.47% longer

Table 2: Supporting CPU comparisons, with each effect tied to its own matched block. Complete source populations, resource eligibility, failures and native response exceptions remain in Appendix A. Between-run variation is descriptive, not a confidence interval or cross-row ranking.

Whole corpus time (s)	Run 1	Run 2	Run 3
Native warm pool	310.478	311.544	318.572
Native snapshot/cache	319.524	320.059	331.712
Packed CPU control	319.510	326.852	321.050
GPU provider	315.870	322.891	323.607

Table 3: Complete source verification for 5,952 ProofNet-lineage attempts per condition. All twelve primary conditions finish with identical basic outcomes within each matched block. Four separate resource profiles also complete. No complete miniF2F GPU comparison is available.

complete Lean kernel on the device.

Across each primary repetition, retained counters record 2,201 supported operations out of approximately 2.38 million provider calls, corresponding to 0.0925%. The percentage measures coverage by call count, not the fraction of verification time that could be accelerated. Each GPU condition has one failed worker without terminal counters, so the totals are observed rather than exhaustive. Per-process cumulative maxima give summed provider wall times of 3.169, 4.921 and 6.158 seconds, including initialization sums of 1.326, 1.178 and 1.274 seconds. These timers include waits and overlap across workers. Subtracting them from whole-corpus makespan would not identify the cause of the slowdown.

This implementation adds host admission, packet preparation, transfers and synchronization while leaving most calls on the native path. The complete results reject an acceleration claim for this implementation on this cohort. They do not establish whether GPU sharing caused the slowdown. No matched co-tenant activity comparison isolates that effect. **[RESULT: Complete miniF2F comparison, cost-weighted supported-work coverage and remaining strongest-checker comparisons.]**

Four separately instrumented complete conditions observe whole times of 328.046 seconds for warm execution, 348.390 for snapshot/cache, 338.670 for the packed CPU control and 342.949 for GPU execution. Their sampled fleet PSS maxima are respectively 5,289,626, 9,268,751, 5,161,250 and 6,315,211 KiB. These are single instrumented conditions. One-second non-atomic sampling misses short-lived processes and peaks, and the sampling process contributes CPU overhead. These clocks do not replace the primary comparisons or establish replicated memory uncertainty. GPU context and driver memory, as well as other owners' memory, are outside host-PSS figures.

Answering RQ4 at its stated scope requires the remaining workload and strongest-checker comparisons. RQ4 remains unanswered until measured complete boundaries establish a device benefit or a negative opportunity bound.

6.6 Limitations

The service applies at supported native boundaries and retains the verifier as the authority. Supported backend versions and mutable runtime extensions constrain deployment. Logical equivalence alone does not promise equal resource-bounded responses. Wall-time limits depend on machine load and execution speed, so the evaluation states their operational scope separately from deterministic work limits. Valid negative and mixed results bound the claimed operating range.

Public generated attempts are replay workloads, whose arrivals can differ from a deployed multi-user service. Fixed published proof corpora remain useful controls but do not establish real candidate fan-out. Full GPU Lean checking is a distinct result from acceleration of a metadata primitive, and GPU certificate proving is a distinct workload from ordinary proof checking.

7 Discussion

Separating preparation from candidate execution connects native behavior to service resource use. Sharing preparation can reduce replication, while each candidate's mutation and lifetime determine how much of that sharing survives. The matched comparison therefore accounts for behavior, cancellation and fleet memory together, rather than treating retained context alone as the service benefit. Extending the service to another verifier requires a supported runtime boundary and qualification against its original interface.

8 Related work

Persistent verification state. Persistent Lean environments and native proof-state snapshotting retain elaborated context across attempts [11, 36]. Stock Lean also saves full elaboration or post-import snapshots [13], while Kimina provides bounded warm verification with environment reuse [32, 34]. These mechanisms provide the direct comparison for preparation reuse. The service evaluates independent native execution, cancellation and bounded fleet resources in addition to logical environment branching. This distinction requires matched evidence, rather than a novelty claim for snapshots themselves.

LeanPolish already restores native tactic state, resumes unchanged commands from saved checkpoints and supports persistent import workers [18]. Its released proof-improvement attempts include failed alternatives, while menu screening and later source verification have different boundaries [1]. These existing mechanisms strengthen the native-reuse comparison. Failure collection and saved-state execution are established baseline capabilities. LeanTree factorizes proof states according to metavariable dependencies, supporting state reuse and parallel proof search [9]. Independent proof goals and reusable intermediate states are therefore established mechanisms; a new service must demonstrate its additional behavior and resource benefit.

Axle combines preloaded Mathlib with per-request processes [38]. Its published normalized-sample comparison measures computation rather than the complete public service path. Isolated preloaded serving is therefore existing work, and our complete behavior and resource comparison with unchanged sources must establish any additional benefit.

Verification caching. Fine-grained verification caching avoids rechecking unchanged program re-

gions [19]. The service comparison combines competent caching with warm execution, so cold startup cannot account for the entire claimed advantage. Caching and session reuse retain different parts of verification work. Native session reuse retains incremental command history and requested-artifact state [40]. Reconstruction with equivalent assertions remains a different execution history from the original native stream.

Prepared execution. Prepared execution and copy-on-write sharing are established systems mechanisms. SOCK and SEUSS share prepared state to reduce startup and increase parallel service density [31, 2]. Spice reduces disk snapshot restoration costs by co-designing the storage layout and kernel restore mechanisms [7]. A contribution beyond these prepared-execution systems requires more than specialization to verification. Our comparison tests whether native behavior and bounded shared preparation together require and benefit from a verification-specific execution mechanism.

Heterogeneous checking. GPU and CPU checker representations address different parts of verification. Nanoclo uses delayed substitutions, and nanoclo-fortran stores kernel objects in parallel arrays with integer identifiers [16, 15]. These checkers are direct competitors to representation-only acceleration. Our heterogeneous evaluation asks about the expensive dynamic checking that remains and complete verification costs, rather than extrapolating from input graph width.

9 Conclusion

Native preparation and candidate execution history are separate resources in verification serving. Independent branches retain preparation while isolating candidate lifetimes, and their value depends on behavior fidelity and the equal-resource service frontier. [**RESULT: Final findings answering all four RQs, including the measured heterogeneous scope.**] The matched comparison determines whether branching supplies a systems benefit beyond existing warm reuse.

References

[1] Pauline Bourigault. Leanpolish: Verified supervision for lean proof compression, 2026. arXiv preprint, version 1.

[2] James Cadden, Thomas Unger, Yara Awad, Han Dong, Orran Krieger, and Jonathan Appavoo. Seuss: Skip redundant paths to make serverless fast. In *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys)*, 2020.

[3] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction (CADE 28)*, pages 625–635, 2021.

[4] Goedel-Prover contributors. Goedel-prover batch verification workflow, 2026. Source revision d80349d93a80305765492014f0b395cec46e5023; accessed 2026-10-15; year denotes access year.

[5] Goedel-Prover contributors. Goedel-prover inference workflow, 2026. Source revision d80349d93a80305765492014f0b395cec46e5023; accessed 2026-10-15; year denotes access year.

[6] Goedel-Prover contributors. Goedel-prover proofnet dataset, 2026. Source revision d80349d93a80305765492014f0b395cec46e5023; accessed 2026-10-14; year denotes access year.

[7] Ben Holmes, Baltasar Dinis, Lana Honcharuk, Adam Belay, and Joshua Fried. Rethinking process snapshots for Near-Warm serverless cold starts. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*, pages 2007–2025. USENIX Association, 2026.

[8] Q. Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018.

[9] Matěj Kripner, Michal Šustr, and Milan Straka. LeanTree: Accelerating white-box proof search with factorized states in Lean 4. In *2nd AI for Math Workshop at ICML*, 2025. Workshop paper; arXiv:2507.14722v1.

[10] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. Verus: Verifying rust programs using linear ghost types. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1), 2023.

[11] Lean community. Lean community repl, 2026. Accessed 2026-10-07; year denotes access year.

[12] Lean community. Mathlib, 2026. Accessed 2026-10-07; year denotes access year.

[13] Lean contributors. Lean 4.32.0 release: incremental snapshots, 2026. Accessed 2026-10-07; year denotes access year.

[14] Lean contributors. Lean kernel arena, 2026. Accessed 2026-10-07; year denotes access year.

[15] Lean Kernel Arena. Nanoclo-fortran: Lean kernel arena checker, 2026. Accessed 2026-10-07; year denotes access year.

[16] Lean Kernel Arena. Nanoclo: Lean kernel arena checker, 2026. Accessed 2026-10-07; year denotes access year. Checker modifications written by Claude, directed by Sebastian Graf; upstream components credited on the page.

[17] Lean project contributors. Lean 4.27.0: Meta expression definitional equality source, 2026. Meta definitional equality implementation; version v4.27.0; accessed 16 October 2026; year denotes access year, not publication date.

[18] LeanPolish contributors. Leanpolish source implementation, 2026. Revision 70cda81a15cca1f805885df197361102f0123cbb; accessed 2026-10-09.

[19] K. Rustan M. Leino and Valentin Wüstholz. Fine-grained caching of verification results. In *Computer Aided Verification (CAV), Part I*, pages 380–397, 2015.

[20] Linux kernel contributors. The /proc filesystem, 2026. Accessed 2026-10-07.

[21] Linux man-pages project. fork(2) linux manual page, 2026. Accessed 2026-10-07; year denotes access year.

[22] Linux man-pages project. getrusage(2): Linux manual page, 2026. Accessed 15 October 2026; year denotes access year.

[23] Linux man-pages project. sched_setaffinity(2) linux manual page, 2026. Accessed 2026-10-09.

[24] Linux perf contributors. perf-report(1): Linux manual page, 2026. Children and Overhead calculation; accessed 16 October 2026; year denotes access year, not publication date.

[25] lucifer1004. farkas. Source repository, 2026. Revision 5a38a7e; accessed 16 October 2026; year denotes access year, not publication date.

[26] Microsoft Research. Verusage-bench, 2026. Accessed 2026-10-07; year denotes access year.

[27] Microsoft Research and AutoVerus contributors. AutoVerus: Artifact evaluation, 2026. Source revision cbf9c0c6337b224fd8e5b7cb4e01ae65c0f98bc1; accessed 16 October 2026; year denotes access year.

[28] mike1729 and dataset contributors. Goedel workbook migrated to lean 4.27, 2026. Source revision 3c2afe6bfc6bdeca6b78d1f257468164bba56966; accessed 2026-10-15; year denotes access year.

[29] NVIDIA. CUDA programming guide: Intro to CUDA C++, 2026. Accessed 16 October 2026; year denotes access year, not a measured CUDA version.

[30] NVIDIA contributors. Nvidia container toolkit, 2026. Source revision a672e378ef91bea7dfe92c0ed274847fa1cb9236; accessed 2026-10-09.

[31] Edward Oakes, Leon Yang, Dennis Zhou, Kevin Houck, Tyler Harter, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Sock: Rapid task provisioning with serverless-optimized containers. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pages 57–70. USENIX Association, 2018.

[32] Project Numina. Kimina lean server, 2026. Accessed 2026-10-07; year denotes access year.

[33] ProofNet contributors. Proofnet benchmark source, 2026. Accessed 2026-10-14; year denotes access year.

[34] Marco Dos Santos, Hugues de Saxcé, Haiming Wang, Ran Wang, Mantas Bakšys, Mert Ünsal, Junqi Liu, Zhengying Liu, and Jia Li. Kimina lean server: A high-performance lean server for large-scale verification. In *Fifth Workshop on Mathematical Reasoning and AI (MATH-AI) at NeurIPS*, 2025.

[35] SGLang contributors. SGLang: inference framework, 2026. Accessed 16 October 2026; year denotes access year.

[36] Austin Shen and Yunong Shi. Keep the proof state live: Snapshotting for efficient tactic search in lean 4, 2026.

[37] vLLM contributors. vLLM: inference and serving library, 2026. Accessed 16 October 2026; year denotes access year.

[38] Jimmy Xin, Alex Schneidman, Chris Cummins, Karun Ram, Srihari Ganesh, and Jannis Limperg. AXLE: A cloud infrastructure for lean 4 theorem proving utilities, 2026. Version 1; accepted at the 3rd AI for Math Workshop, ICML 2026.

[39] Chenyuan Yang, Natalie Neamtu, Chris Hawblitzel, Jacob R. Lorch, and Shan Lu. Verusage: A study of agent-based verification for rust systems, 2025.

[40] Z3 contributors. Z3 c api, 2026. Accessed 2026-10-07; year denotes access year.

[41] Z3 contributors. Z3 guide: Parameters, 2026. Accessed 2026-10-07; year denotes access year.

[42] Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. miniF2F: A cross-system benchmark for formal olympiad-level mathematics. In *International Conference on Learning Representations*, 2022. Published conference version; author preprint first appeared in 2021.

A Matched CPU comparisons

These supporting comparisons preserve each workload, paired reference, resource configuration, native outcome exception and sampling limitation. Their separate blocks do not establish a global CPU ranking or a heterogeneous speedup.

Complete CPU compilation pairs. A separate matched four-CPU block checks every one of the 29,750 released GoedelWorkbook sources with the original GC service and the same service loading native-compiled Mathlib modules [3]. Whole elapsed time is 32,945.98 versus 15,835.68 seconds in the first pair and 38,443.59 versus 18,306.13 seconds in the reversed-order second pair. These are reductions of 51.93% and 52.38%. The third pair takes 32,484.60 versus 13,149.05 seconds, a reduction of 59.52%. The third pair retains 23 native/basic outcome changes, with ten complete outcomes becoming operational and thirteen operational outcomes becoming complete. No direct complete/rejected transition is observed. Runtime import and library loading are charged, while software build is excluded. In the first pair, native fields match on 29,725 sources after excluding the adapter timer. Sixteen operational outcomes become complete and nine complete outcomes become operational. The second pair has 21 operational-to-complete transitions and 12 reverse transitions. One additional source is operational in both but changes from a decoder failure to a timeout. The other 29,716 native payloads match except the adapter timer. No complete/rejected flip is observed, and failures remain charged. All three primary pairs finish, while the two separate resource profiles remain unfinished. These paired observations strengthen an established CPU baseline. The three observations do not establish a confidence interval or an isolated causal effect under the shared background, a new compiler contribution or GPU acceleration. Clocks from other CPU or oracle blocks are not used as denominators.

A separate producer-compatible Lean4.9 GC comparison processes all 7,808 retained miniF2F source attempts per method with and without selective native Linarith compilation [12]. All three primary pairs finish, taking 12,689.28 versus 11,723.45 seconds, 12,539.74 versus 11,192.98 seconds, and 11,914.91 versus 9,893.72 seconds. Whole elapsed reductions are 7.61%, 10.74% and 16.96%. Their descriptive mean is 11.77%, without a confidence interval. Request p99 rises from 247.85 to 275.35 seconds and from 251.14 to 253.28 seconds in the first two pairs, but falls from 224.27 to 209.72 seconds in the third. The complete count remains 3,008 in all six conditions. The second pair has two operational-to-rejected transitions and five other native diagnostic differences. The third has three such transitions and one operational diagnostic difference. Remaining payloads match except the adapter timer. Lower whole-corpus elapsed time therefore does not establish uniform tail latency, transparent native responses or GPU benefit. The two separate resource conditions also finish, completing the eight-condition acquisition. Their completion does not establish replicated memory savings. The miniF2F comparison uses a different producer version and selective compiled closure from the Goedel block above.

Multiplication-theorem application reuse. We also compare complete verification with and without reuse of resolved multiplication-theorem applications within each nonlinear arithmetic preprocessing invocation. Against the same FarkasFast/native-hybrid route, three complete 29,750-source pairs reduce whole elapsed time from 11,629.90 to 10,487.44 seconds, 12,852.84 to 11,208.51 seconds and 14,753.25 to 12,483.33 seconds. The observed reductions are 9.82%, 12.79% and 15.39%, with a descriptive mean of 12.67%. This observed range is not a confidence interval. Closed-loop request p99 also falls in each pair, from 10.56 to 10.09, 12.16 to 10.73 and 14.45 to 12.00 seconds. These are response distributions within the measured blocks, not an offered-load capacity result. The first Fast/hybrid pair also retains one operational decoder failure that becomes native rejection. This differs from a direct complete/rejected transition and from the original-oracle product comparison below. The third pair retains six operational/completion transitions and otherwise matching native payloads. No complete/rejected transition occurs.

With the original arithmetic oracle retained in both methods, all three primary pairs take 15,385.79 versus 15,475.96 seconds, 16,245.63 versus 15,415.52 seconds, and 14,942.76 versus 12,805.48 seconds with reuse. Whole time is 0.59% longer, then 5.11% and 14.30% lower. The descriptive mean reduction is 6.28%, without a confidence interval. The third pair has five operational-to-complete and five reverse transitions. Other native payloads match except the adapter timer. No complete/rejected transition occurs. The original-oracle pairs differ in the direction of their elapsed-time changes. The different blocks and shared background do not isolate a causal oracle interaction or establish a uniform original-route gain.

Two separate Fast/hybrid resource captures complete the eight-condition comparison. Observed sampled peak PSS falls from 7,674,160 to 6,961,995 KiB, with incomplete reads in 33 of 7,051 and 34 of 6,198 non-atomic samples. These single profiles do not establish exact peaks or replicated memory savings. Seven operational/completion transitions remain. Both original-oracle resource profiles finish, completing the eight-condition comparison. Their sampled peak fleet PSS rises from 7,486,707 to 8,005,904 KiB. These single non-atomic profiles do not establish exact peaks or replicated memory savings.

These CPU observations concern an ordinary optimization within its matched block. They do not estab-

lish the globally strongest CPU method, systems novelty or GPU acceleration.

Complete native arithmetic primary series. Three matched Goedel CPU pairs compare compiled Mathlib with the same service using the native exact arithmetic oracle. Reference whole times are 10,531.82, 13,112.67 and 15,097.58 seconds. Candidate times are 11,030.02, 10,666.44 and 14,886.85 seconds. The candidate takes 4.73% longer, then 18.66% and 1.40% less time. The descriptive mean reduction of 5.11% is not a confidence interval or a stable general advantage. The direction of the timing change differs across pairs, and reference times vary even with the same code. Each condition returns all 29,750 inputs. The third pair retains 13 native outcome differences, all involving an operational outcome, with no direct complete/rejected transition. Operational failures stay charged. All eight conditions, including the two separate resource profiles, finish. In the single profile pair, observed sampled peak PSS rises from 6,561,664 to 7,478,115 KiB. Incomplete, non-atomic sampling does not establish exact or replicated peaks. This pair shows no measured memory saving. These complete comparisons preserve a competent CPU competitor. They do not establish the globally strongest CPU path or heterogeneous acceleration.

For the Fast/hybrid product-reuse and native exact-oracle comparisons, we divide each candidate's whole time by its own paired reference time. A ratio below one means lower elapsed time. Figure 4 displays all three primary observations per comparison. Fast/hybrid product reuse has the same direction in every pair, while the native exact oracle has both increases and reductions in elapsed time. The rows use different references and CPU eligibility, so they do not rank methods across comparisons.

Numerical support search and exact arithmetic. Three balanced primary trios compare tiered exact arithmetic, native FP64 hybrid search and HiGHS support search with exact repair over all 29,750 sources per method. They use the same native-compiled library closure, four outstanding requests and shared CPU eligibility 4–7. Relative to tiered, hybrid increases whole elapsed time by 10.61%, 5.22% and 23.43%. HiGHS takes 11.83% longer, then 11.45% less time, then 21.47% longer. These observations retain tiered as a necessary CPU competitor. The numerical paths do not establish a consistently faster replacement. They do not rank different CPU compositions. Native diagnostics also differ. One source changes from complete to rejected under HiGHS in every trio, with a recursion-depth diagnostic. All failures remain charged. All twelve conditions finish, including three separate resource profiles. Their sampled peak fleet PSS is 7,005,488 KiB for tiered, 7,668,612 KiB for hybrid and 8,723,814 KiB for HiGHS. The HiGHS capture has 1,448 incomplete samples out of 6,582. One-second non-atomic sampling and one profile per method do not establish replicated memory or CPU superiority. The recorded rusage counter includes observer, self and waited children, without validated exhaustive descendant coverage. Shared background limits causal and global CPU conclusions.

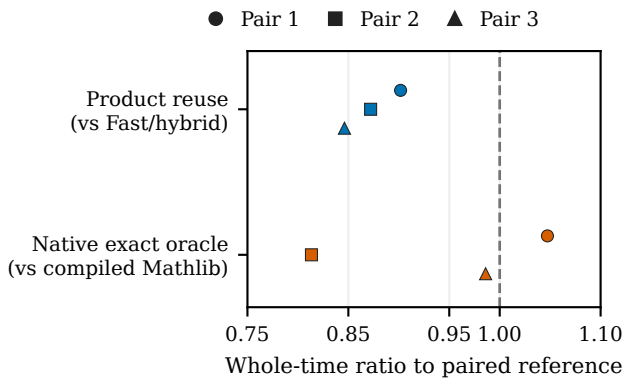


Figure 4: Whole-time ratios for two complete CPU comparisons, each processing all 29,750 GoedelWorkbook sources per condition. Each marker is one complete primary pair normalized to its own reference. The dashed line is parity. All three observations show between-pair variation, not confidence intervals. The ratio axis zooms around parity to make that variation visible. Fast/hybrid product reuse uses shared CPU eligibility 0–23, and the native exact oracle uses 0–3. Different references and eligibility prevent cross-row ranking. Separate resource profiles are excluded and operational failures remain charged. Fast/hybrid product reuse is consistently faster in these pairs, while the oracle comparison is mixed.

Full and restricted numerical preprocessing. Three rotated four-method blocks complete all 29,750 sources per condition, comparing Full and Fast preprocessing with native hybrid and HiGHS support search. All twelve primary conditions use shared CPU eligibility 16–19 and four outstanding HTTP requests; separate resource profiles are excluded from these primary timing comparisons. Fast changes whole elapsed time by −4.52%, +8.50% and −5.07% relative to Full with hybrid, and by +7.87%, +6.12% and −4.44% with HiGHS. It does not show a stable benefit across these observations. HiGHS takes 10.47%, 25.14% and 12.01% longer than hybrid under Full, and 24.80%, 22.39% and 12.76% longer under Fast. Basic outcomes differ for 9–15 sources per paired comparison. Each HiGHS/hybrid pair includes the same complete-to-rejected source with a recursion-depth diagnostic; operational failures and all diagnostics remain charged. These are matched composition-specific elapsed observations, not transpar-

ent replacements, confidence intervals, a global CPU ranking or GPU evidence. Shared background and sequential acquisition remain limitations.

Complete original-path cost profile. A separate instrumented capture returns all 29,750 GoedelWorkbook4.27 sources and retains body profiles for 29,689 of them. All 61 sources without a body profile have operational outcomes. The arithmetic prover `linarith` [12] and its detailed regions have the largest recorded wall-time totals for leaf regions, followed by `simp`. These thresholded trace weights measure wall intervals that can overlap across threads and requests, rather than CPU fractions or device-eligible work. The observation characterizes the original arithmetic route. Residual costs after the strongest CPU route remain to be measured.

Complete compiled-route CPU sampling. A separate instrumented compiled-route run returns all 29,750 sources and retains 5,401,763 user-CPU samples with zero reported lost samples. Exclusive sample counts and periods reconcile across flat and callchain reports. Standard `perf` children weights include callees and overlap [24]. The native `importModules` entry displays 0.36% against all recorded user-CPU period. This entry does not cover all preparation, initialization or elapsed I/O, and does not bound cold-to-warm savings. Against the same recorded user-CPU period, the displayed inclusive weights are 35.00% for the definitional-equality export in Lean's Meta layer, 16.96% for a sparse-simplex certificate-oracle entry and 10.81% for a typeclass-synthesis entry. The first symbol is exported by a `MetaM` implementation in the installed Lean4.27 source [17]. It is not a pure C++ kernel checking fraction. These entries cannot be summed as disjoint phases. Separately, reference-count decrement and small allocation account for 20.05% of native exclusive recorded period. That pair is not an exhaustive runtime category. Sampling, unknowns and callchain limits remain, and neither view supplies a decomposition of complete-verification wall time, a GPU ceiling or batching readiness. This capture characterizes the compiled original arithmetic route. Residual costs after selecting a different strongest CPU composition remain open.

Independent argument-comparison opportunity. The same sampled capture displays 23.16% inclusive CPU weight for argument comparison, including stateful work. A separate instrumented capture attributes 23.608936% of completed observed outer calling-thread CPU to entries without expression or universe metavariable flags. These quantities have different captures and scopes. Neither measures useful independent argument work or complete-verification critical-path coverage. Multiplying cross-capture fractions supplies only conditional CPU-bound sensitivity scenarios, not an end-to-end bound. The batch interface therefore has no admitted GPU provider. Another narrow backend requires evidence of useful independent work with complete setup, transfer and fallback costs.